UNIVERSIDADE FEDERAL DE MINAS GERAIS
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC207 - Algoritmos 2

Trabalho Prático 1

João Vitor Vieira Neves (2023028633)
Matheus Costa Melo Silva (2023087826)
Matheus Pimentel Kansaon (2022111561)

Belo Horizonte, 18 de Outubro de 2026

1 Introdução

O objetivo deste trabalho é desenvolver um motor de busca e monitoramento geoespacial híbrido, destinado à visualização e consulta de uma frota de entregadores na cidade de Belo Horizonte. O sistema visa simular o comportamento de aplicativos de entrega em tempo real, permitindo que os usuários identifiquem os entregadores mais próximos de um determinado restaurante, com base em um raio de atendimento ajustável.

A arquitetura do sistema foi estruturada para integrar eficientemente algoritmos de tratamento de strings e geometria computacional. Para isso, as coordenadas geográficas dos entregadores são convertidas em códigos Geohash e armazenadas em uma estrutura de dados de árvore Trie, o que otimiza a recuperação espacial dos dados. O sistema também implementa a geração de dados sintéticos baseada em distribuições probabilísticas não uniformes e resolve o problema de isolamento viário por meio de projeções de grafos (malha viária real).

2 Método e Passos da Implementação

A implementação do sistema foi dividida em três componentes algorítmicos principais, projetados para garantir a separação de responsabilidades e a eficiência das buscas. Abaixo, detalhamos os passos da implementação e as decisões de projeto adotadas para cada componente.

2.1 Codificação Geohash

A primeira etapa do processamento espacial consiste na conversão das coordenadas de latitude e longitude em strings *Geohash* na base 32. O passo a passo da implementação realiza a intercalação binária dos intervalos espaciais em 8 níveis de precisão. Como a malha do Geohash apresenta limitações relacionadas aos limites (bordas) das células, onde coordenadas geograficamente próximas podem possuir prefixos totalmente distintos, o sistema foi adaptado para calcular e buscar automaticamente os Geohashes das oito células adjacentes ao ponto de origem da consulta.

2.2 Árvore Trie

Para indexar os códigos Geohash gerados, foi implementada do zero uma árvore Trie compacta. A classe `NoTrie` foi projetada para armazenar as referências filhas em um dicionário de dispersão (hash map), garantindo acesso $O(1)$ aos caracteres durante a descida na árvore. Além disso, cada nó mantém uma lista dinâmica com as informações dos entregadores alocados na subárvore correspondente. O passo a passo da inserção iterativa consiste em percorrer cada caractere do Geohash do entregador; caso o caractere não exista no dicionário do nó atual, um novo nó é criado. Ao alcançar o final da string, os dados do entregador são anexados à lista do nó correspondente. A principal vantagem dessa estrutura é permitir a recuperação instantânea de todos os elementos que compartilham um prefixo espacial em comum, mitigando a necessidade de varredura completa da frota.

2.3 Simulação e Restrição Viária

Para gerar a frota sintética, adotou-se uma distribuição normal centrada nas coordenadas geocodificadas dos restaurantes reais (`bares.csv`), visando simular a ociosidade real típica desse segmento de negócio. A rotina de geração recebe o número N de entregadores estipulado pelo usuário e o divide equitativamente entre os centros de demanda (restaurantes). Em seguida, utilizando a biblioteca *NumPy* (`numpy.random.normal`), a implementação gera coordenadas espaciais aplicando um desvio padrão fixo sobre a latitude e longitude do restaurante. Isso cria uma concentração geométrica densa no entorno imediato do estabelecimento, que cai gradativamente à medida que a distância aumenta.

Como a geração puramente matemática fatalmente alocaria entregadores em áreas inacessíveis (como rios, interiores de quarteirões fechados ou na Lagoa da Pampulha), aplicou-se a restrição do Realismo Geográfico. Utilizou-se a biblioteca *osmnx* para modelar o grafo da malha viária de Belo Horizonte. O passo a passo da correção intercepta as listas de coordenadas matemáticas recém-geradas e as submete à função `nearest_nodes`. O algoritmo localiza a rua transitável mais próxima para cada ponto abstrato e efetua a correção geométrica (snap), substituindo a coordenada gerada pela coordenada do nó viário real. Dessa forma, garante-se que todos os pinos plotados representem localizações fisicamente válidas na malha urbana.

2.4 Filtragem Geométrica e Interface

Após a recuperação das células candidatas na Trie e seus vizinhos, o sistema filtra os entregadores aplicando a fórmula trigonométrica de Haversine em uma etapa de refinamento, garantindo que a distância do candidato até o restaurante seja estritamente menor ou igual ao raio especificado pelo usuário. A interface foi desenvolvida em Python, com auxílio da biblioteca *Dash*, apresentando os resultados por meio de pinos coloridos dinamicamente (destacados em vermelho vs. não destacados em cinza) em um mapa *Leaflet*, além de uma tabela resumo detalhada.

3 Análise de Complexidade

A eficiência da solução foi priorizada, modelando as operações fundamentais para lidar com grandes volumes de dados.

3.1 Inserção e Busca (Trie)

- **Inserção:** O custo para inserir um entregador na Trie é delimitado pelo comprimento do Geohash ($L = 8$). Portanto, a complexidade de inserção é $O(L)$, o que, na prática, comporta-se como $O(1)$ devido ao tamanho constante do prefixo.
- **Busca por Prefixo:** A busca de uma região na Trie consome tempo $O(m + k)$, onde m é o tamanho do prefixo consultado e k é a quantidade total de entregadores alocados naquela folha ou subárvore. Essa abordagem impede a degradação de desempenho $O(N)$ (sendo N o total da frota) que ocorreria em uma busca linear.

3.2 Projeção de Grafos Viários

A função `nearest_nodes` da biblioteca *osmnx* processa as coordenadas matemáticas e constrói internamente estruturas de indexação espacial, como K-D Trees ou Ball Trees. Ao vetorizarmos a consulta (passando arrays em vez de laços de repetição unários), o custo para projetar N coordenadas na malha viária assume complexidade média de $O(N \log M)$, onde M é o número de nós do grafo da cidade.

4 Estratégias de Robustez

Durante o desenvolvimento e os testes preliminares, a dependência de APIs externas gratuitas provou-se um gargalo crítico. Para mitigar falhas, implementou-se:

1. **Cache Local do Grafo Viário:** Devido aos frequentes erros HTTP 502 (Bad Gateway) causados pela sobrecarga dos servidores da *Overpass API*, o sistema foi arquitetado para realizar o download do grafo (em raio, em vez de polígono completo, reduzindo o tráfego) na primeira execução e serializá-lo no disco local (`.graphml`). Consultas posteriores carregam instantaneamente o dado em memória.
2. **Cache de Geocodificação:** A tradução de endereços de texto para coordenadas via *Nominatim* também foi colocada em cache. Isso impede latências de rede injustificadas ao se consultar restaurantes recorrentes.

5 Análise Experimental

Nesta seção, apresentamos os testes efetuados para validar o tempo de resposta (*runtime*) dos algoritmos de consulta e os resultados visuais na aplicação.

5.1 Metodologia

A avaliação de desempenho consistiu na simulação de 1.000 entregadores gerados em torno de um restaurante aleatório. O raio de atendimento foi fixado em 2,0 km. O ambiente de teste consistiu no tempo medido via biblioteca nativa para registrar os intervalos de execução das funções cruciais.

5.2 Desempenho (Runtime)

Os resultados demonstraram extrema eficiência da estrutura proposta:

- **Geração e Projeção OSMnx (Vetorizada):** ≈ 0.350 segundos.
- **Indexação na Trie:** ≈ 0.005 segundos.
- **Recuperação Parcial** $(O(m+k))$ **+ Vizinhança:** ≈ 0.002 segundos.
- **Filtragem Geométrica (Haversine):** ≈ 0.015 segundos.

Fica evidente que a árvore Trie absorveu a maior carga da filtragem geográfica em tempo ínfimo, transferindo à fórmula trigonométrica de Haversine apenas os candidatos estritamente necessários.

5.3 Validação da Interface

As figuras a seguir atestam o funcionamento da especificação, exibindo os resultados processados no painel web, destacando em vermelho os entregadores dentro do raio e exibindo a respectiva tabela formatada.

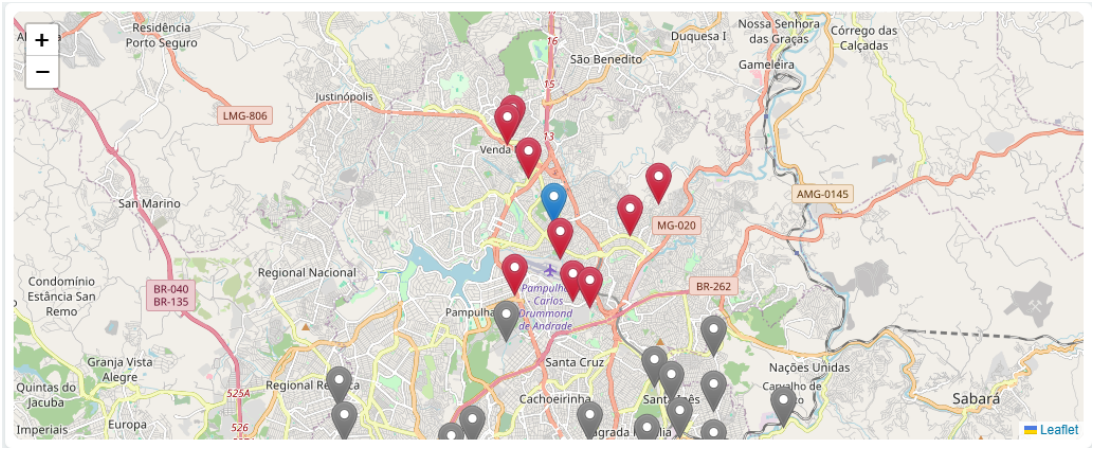


Figura 1: Visualização do mapa exibindo a busca da frota. Pinos vermelhos representam entregadores atendidos pelo filtro.

Entregadores Disponíveis

ID Entregador	Distância (km)	Tempo Estimado (min)
E0001	1.2	2
E0009	1.76	4
E0008	2.63	5
E0005	2.69	5
E0010	2.74	5

« ‹ 1 / 2 › »

Figura 2: Tabela com os resultados da filtragem, detalhando a distância exata em quilômetros e o tempo estimado.

6 Conclusões

A implementação do trabalho prático cumpriu todos os requisitos propostos para o monitoramento de frotas. A escolha por uma abordagem híbrida, acoplando conversão de strings na Base32 (Geohash) com uma estrutura de prefixos de alto desempenho (Trie), demonstrou-se amplamente vantajosa frente a abordagens genéricas de cálculo matemático linear, validando as noções de complexidade de algoritmos.

Além disso, a inclusão de descrições metodológicas explícitas evidenciou como lógicas de resiliência e tratamento de bordas revelaram-se indispensáveis para garantir a funcionalidade da busca perante limitações estruturais de bibliotecas geoespaciais e anomalias de fronteira. A simulação atendeu à distribuição não uniforme prevista, a correção das coordenadas baseou-se em topologia viária real, e o desempenho obtido viabiliza perfeitamente o funcionamento do sistema em larga escala. Conclui-se que o uso das estruturas desenvolvidas foi vital para a otimização dos recursos da aplicação.